

Relatório de Auditoria de Segurança

— micaeltatuagem/tatuagem

Data da auditoria	28 de agosto de 2026
Commit auditado	d501992272ac95a7876c41eb549a35ccdfefde59
Repositório	github.com/micaeltatuagem/tatuagem (público)
Tipo de auditoria	Revisão estática de código-fonte (SAST manual)

Escopo auditado

Repositório público micaeltatuagem/tatuagem (branch main, commit d501992), clonado em ambiente isolado no estado exato publicado no GitHub. Escopo: 2.700 arquivos versionados — site estático (GitHub Pages), 11 painéis administrativos client-side, o proxy Cloudflare Worker de escrita no GitHub (worker-proxy/), os dois workflows de GitHub Actions (.github/workflows/) e o script Node de geração estática do Guia (scripts/generate-guia-pages.js). Não incluído no escopo (não acessível a partir do repositório): políticas RLS configuradas diretamente no dashboard do Supabase, e o código das RPCs Postgres chamadas via supabase.rpc(...).

Nota metodológica

Stack detectada: site estático hospedado no GitHub Pages, sem framework de frontend, sem servidor de aplicação e sem build step — qualquer commit em main é publicado direto. Persistência de dados via Supabase (Postgres + PostgREST + Auth + Storage), usado só pelos sistemas de cadastro/indicação/anamnese/layaway e pelo Guia; o catálogo de flash é 100% arquivo JSON estático. Escrita no repositório GitHub feita pelos painéis admin através de um proxy Cloudflare Worker dedicado (tatuagem-gh-proxy), que guarda o token real do GitHub e nunca o expõe ao navegador. Mapeamento das 5 categorias pedidas para essa stack: (1) 'banco sem tranca' mapeado para políticas RLS do Supabase e para validação de regra de negócio em INSERTs diretos das páginas públicas; (2) 'permissão definida no navegador' mapeado para os dois mecanismos de login dos painéis admin (senha via Worker autenticado vs. Supabase Auth real vs. comparação só no JavaScript do navegador); (3) IDOR mapeado para RPCs Supabase que aceitam um identificador (código/telefone) sem verificar posse; (4) 'chaves expostas' mapeado para segredos em código-fonte, histórico git e workflows de CI; (5) XSS mapeado para innerHTML/href com dados de banco/URL não sanitizados, e para a geração estática do Guia.

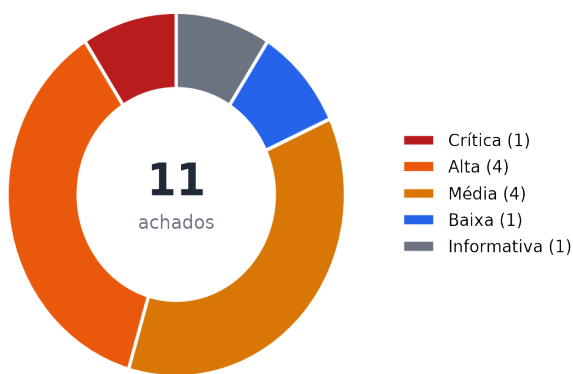
Este relatório foi gerado por uma revisão manual assistida do código-fonte publicado no repositório. Achados são reportados apenas quando verificados diretamente no código; onde a verificação completa dependia de configuração externa ao repositório (ex.: políticas RLS do Supabase), isso é declarado explicitamente em cada achado.

Resumo executivo

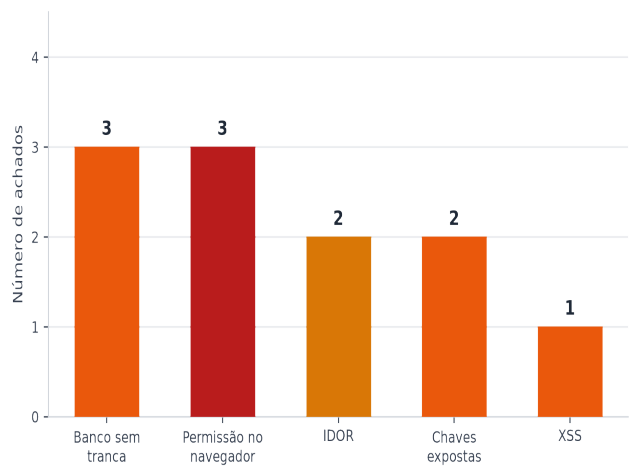
✓ **Status pós-auditoria:** C1, A2, A4 e M3 já foram corrigidos (código no commit d2308a8f; políticas RLS de guia_verbetes e clientes_promo revisadas e confirmadas em produção em 29/08/2026). M4 foi verificado e não era uma vulnerabilidade. Pendente: confirmar a política RLS do bucket de Storage guia-imagens (parte do achado C1) e as recomendações de prioridade P2/P3 do relatório.

1	4	4	1	1
Crítica	Alta	Média	Baixa	Informativa

Achados por severidade



Achados por categoria



O achado crítico (C1) e três dos quatro achados de severidade alta (A2, A4, e parte de A1) formam uma cadeia única: a ausência de autenticação real no painel do Guia habilita escrita irrestrita em guia_verbetes, que por sua vez alimenta um path traversal na geração estática e um vetor de XSS via href — corrigir C1 neutraliza a explorabilidade prática de A2 e A4, mas ambos devem ser corrigidos de forma independente (defesa em profundidade).

Pontos fortes

O que foi verificado no código e está protegido corretamente — evidencia a cobertura desta auditoria.

- **Worker gh-proxy nunca expõe o token real do GitHub ao navegador**
worker-proxy/worker.js: o token real (GITHUB_TOKEN) só existe como secret do Cloudflare (linha 61); a senha é conferida no servidor do Worker (linha 38), e há uma trava de path que só libera chamadas para o repositório esperado (linhas 46-53).
- **Seis painéis admin passam pelo Worker autenticado antes de qualquer escrita**
adminflash.html, adminblog.html, adminaerografia.html, admincorpos.html, admin Galeria.html e admin-hub.html verificam a senha contra o Worker (que a confere no servidor) antes de liberar GH_TOKEN em sessionStorage — nenhum deles hardcodeia a senha real no HTML.
- **Três painéis usam autenticação real do Supabase (não cosmética)**
admin_promo.html:503, admin_layaway.html:515 e adminanamnese.html:544 usam sb.auth.signInWithPassword(), verificado no servidor do Supabase — ao contrário do padrão cosmético encontrado em adminguia.html (achado C1).
- **Nenhuma chave privilegiada (service_role) exposta em código-fonte**
Busca em todo o repositório (HTML/JS) por padrões de service_role/sb_secret_ não retornou nenhum resultado. A única chave privilegiada usada no projeto (SUPABASE_SERVICE_ROLE_KEY, em .github/workflows/supabase-heartbeat.yml) está corretamente referenciada via \${ secrets.SUPABASE_SERVICE_ROLE_KEY } do GitHub Actions, nunca em texto puro.
- **Histórico do git sem tokens/chaves privilegiadas vazadas**
Busca por padrões de token do GitHub (ghp_/github_pat_), chave secreta do Supabase (sb_secret_) e referências a service_role em todo o histórico (git log --all -p) não encontrou nenhum resultado, além da senha de baixa entropia já coberta no achado A1.
- **Geração estática do Guia escapa todo texto injetado no HTML**
scripts/generate-guia-pages.js define escapeHtml() (linha 42) e aplica em termo, descricao/corpo, imagem_alt e títulos antes de embutir no HTML estático das 389 páginas do Guia — proteção consistente contra XSS armazenado na publicação em massa.
- **Painéis admin escapam consistentemente dados enviados pelo público**
adminanamnese.html, admin_promo.html e admin_layaway.html implementam e aplicam escapeHtml() antes de inserir nome/whatsapp/observações (campos de texto livre vindos de reserva.html e anamnese.html) em innerHTML — protege o painel contra XSS armazenado via os formulários públicos.
- **Nenhum uso de eval, new Function ou document.write em todo o repositório**
Busca por esses padrões em todos os arquivos .html/.js do repositório não retornou nenhum resultado.
- **Formulários públicos de INSERT não fazem leitura cruzada de outros registros**
anamnese.html faz apenas INSERT (linha 861), sem nenhum SELECT de outros registros — confirmado por busca de padrões .from(/.select(/.rpc(no arquivo.

Pontos fracos — riscos centrais

C1	Painel do Guia sem autenticação real; endpoint público sem login algum
A1	Senha admin do Guia exposta permanentemente no histórico git
A2	Path traversal via slug não sanitizado na geração estática do Guia
A3	Sem rate limiting na senha que protege a escrita de todo o repositório
A4	XSS armazenado via URL javascript: em campo gravável sem autenticação
M1/M2	IDOR: enumeração de clientes por código de indicação e por WhatsApp
M3/M4	Regras de negócio e dado sensível (CPF/RG/saúde) dependem de RLS não verificável no repo

Achados detalhados

Cada achado lista arquivo(s) e linha(s) exatos, descrição do problema, impacto, condição de explorabilidade e sugestão de correção. Achados marcados como "item de verificação" não foram confirmados como vulnerabilidade a partir do código-fonte — o repositório não contém as políticas RLS do Supabase, que vivem só no dashboard.

1. Banco sem tranca

[A2] **ALTA** — Path traversal via slug não sanitizado em generate-guia-pages.js

[CORRIGIDO]

✓ Corrigido no commit d2308a8f (slug validado por allow-list + checagem de contenção de path em generate-guia-pages.js).

Arquivo:linha

• scripts/generate-guia-pages.js:529-533

Descrição: v.slug (campo da tabela guia_verbetes — gravável por qualquer um, ver C1) é usado sem sanitização em path.join(OUTPUT_DIR, v.slug) para criar diretório e escrever index.html. Um slug como '.././algum-caminho' resolve para fora de guia/. O último passo do workflow (generate-guia-pages.yml) só faz `git add guia/ sitemap.xml guia.html`, o que limita o que acaba sendo commitado/publicado — mas a escrita em disco fora de guia/ ainda acontece dentro do runner da Action antes desse git add, o que já é uma falha de sanitização que deveria ser corrigida independentemente do RLS (defesa em profundidade).

Impacto: Escrita de arquivo em caminho arbitrário dentro do sistema de arquivos do runner da GitHub Action durante a geração das páginas do Guia. O alcance completo (se atinge arquivos que afetam o commit/push seguinte, como .git/) não foi testado neste código-fonte estático — recomenda-se tratar como severidade alta e corrigir por precaução.

Condição de explorabilidade: Depende de conseguir gravar um slug malicioso em guia_verbetes — ver C1 (hoje, isso é possível sem autenticação).

Sugestão de correção: Sanitizar v.slug com uma allow-list (ex.: /^[a-z0-9-]+\$/) tanto na escrita em disco quanto, idealmente, via constraint/check no banco (CHECK (slug ~ '^[a-z0-9-]+\$')).

[M3] MÉDIA — Falta de validação de regra de negócio no INSERT direto em clientes_promo [CORRIGIDO]

✓ Corrigido no banco (29/08/2026): as políticas frouxas delete_admin/select_admin/update_admin (que liberavam SELECT/UPDATE/DELETE pra qualquer usuário authenticated, não só o admin) foram removidas; o INSERT público foi recriado com WITH CHECK exigindo que status_desconto, tatuagens_feitas, indicacoes_confirmadas, sessao_gratis_liberada, sessao_gratis_usada e as datas de liberação/uso estejam nos valores neutros de início — confirmado via consulta a pg_policies.

Arquivo:linha

• reserva.html:1321 (sb.from('clientes_promo').insert([payload]))

Descrição: O formulário público só envia nome, whatsapp, email, codigo_indicacao, indicado_por e canal (não inclui campos de premiação como sessao_gratis_liberada). Mas como o INSERT vai direto para o Supabase com a anon key, nada no código-fonte do site impede que alguém monte a própria chamada HTTP (fora do formulário) enviando colunas extras nesse mesmo INSERT. A proteção contra isso depende inteiramente de RLS/constraints do lado do Supabase, que não são visíveis neste repositório.

Impacto: Se a tabela não tiver defaults/constraints que bloqueiem sobrescrever colunas de premiação via INSERT, um atacante poderia se autoconceder desconto ou sessão grátis sem cumprir as 5 indicações.

Condição de explorabilidade: Requer montar uma chamada HTTP direta à API REST do Supabase (fora do formulário) — trivial com a anon key pública.

Sugestão de correção: Confirmar no Supabase que colunas de premiação têm DEFAULT/GENERATED de forma que INSERT do role anon não possa sobrescrevê-las (RLS com WITH CHECK explícito por coluna, ou trigger). Idealmente, mover a lógica de premiação para uma RPC SECURITY DEFINER em vez de INSERT direto na tabela.

[M4] MÉDIA — RLS de SELECT em anamneses (CPF/RG/dados de saúde) não verificável a partir do repositório [VERIFICADO OK]

✓ Verificado no banco (29/08/2026) — não era uma vulnerabilidade: RLS de anamneses já restringe SELECT/UPDATE/DELETE ao UUID específico do admin (auth.uid() = '9adc605d-...'), INSERT público só (esperado). Nenhuma ação necessária.

Arquivo:linha

- anamnese.html:836-861 (payload com nome, rg, cpf, endereço, celular, respostas de saúde)
- adminanamnese.html:518, 529-544 (leitura só depois de signInWithPassword — correto no código)

Descrição: anamnese.html só faz INSERT (confirmado — nenhum SELECT de outros registros nessa página). adminanamnese.html só carrega dados depois de autenticação real via Supabase Auth. O código do site está correto nesse ponto. O que NÃO é verificável a partir deste repositório é a política RLS de SELECT da tabela anamneses no lado do Supabase — não há nenhum arquivo .sql/migração no repo. Dado que a tabela guarda CPF, RG, endereço e respostas de saúde (dado sensível pela LGPD), o custo de uma política de SELECT mal configurada (ex.: permitindo anon) é desproporcionalmente alto comparado a outras tabelas.

Impacto: Não confirmado como vulnerabilidade — listado como item de verificação manual prioritária dado o dado sensível envolvido.

Condição de explorabilidade: Não aplicável (item de verificação, não de exploração confirmada).

Sugestão de correção: Confirmar no dashboard do Supabase que a política de SELECT em anamneses permite acesso só ao role authenticated (idealmente restrito por e-mail/role específico do admin, não a qualquer usuário autenticado).

2. Permissão no navegador

[C1] **CRÍTICA** — Painel do Guia sem autenticação real — escrita livre em guia_verbetes e Storage **[CORRIGIDO]**

✓ Corrigido no código (commit d2308a8f) e confirmado no banco: a política "Guia verbetes - escrita publica (anon)" (cmd=ALL, role anon, sem condição) foi removida e substituída por uma política restrita ao role authenticated — confirmado via consulta a pg_policies em 29/08/2026 (só sobraram a política de SELECT público e a nova de escrita authenticated). Pendente apenas confirmar a mesma restrição no bucket de Storage guia-imagens.

Arquivo:linha

- adminguia.html:219-220, 224, 251-259, 383-386, 398-408, 448-465
- admin-guia-upload.html:54-55, 75-95, 126-135 (arquivo público sem NENHUMA tela de login)

Descrição: adminguia.html decide se mostra o painel comparando a senha digitada com uma constante no JavaScript do navegador (ADMIN_SENHA, linha 224) — não existe nenhuma verificação no servidor. Todas as escritas (INSERT/UPDATE/DELETE em guia_verbetes, upload no Storage) usam só a chave publicável (anon) do Supabase, sem nunca chamar supabase.auth. A 'senha' é uma cortina cosmética: dá pra ler o código-fonte da página pública e pegar ADMIN_SENHA, ou simplesmente ignorá-la e chamar a API REST do Supabase direto com a anon key (também pública, embutida em guia.html e guia-verbete.html). admin-guia-upload.html confirma isso na prática: é um formulário público, publicado no ar, que faz o mesmo INSERT sem absolutamente nenhuma tela de login. Efeito em cadeia: guia_verbetes alimenta a GitHub Action generate-guia-pages.yml, que publica automaticamente (commit + push) as mudanças no site ao vivo — ou seja, é também um vetor de defacement/spam automático do site público, não só do banco.

Impacto: Qualquer pessoa na internet pode criar, editar ou apagar verbetes do Guia e subir arquivos no Storage sem autenticação nenhuma, e ver essas mudanças publicadas automaticamente no site público em minutos, via a Action já existente.

Condição de explorabilidade: Nenhuma condição especial — o endpoint admin-guia-upload.html funciona sem senha, feature flag ou configuração adicional.

Sugestão de correção: Trocar a checagem client-side por supabase.auth.signInWithPassword() (mesmo padrão já usado em admin_promo.html/admin_layaway.html/adminanamnese.html) e configurar RLS em guia_verbetes para só permitir INSERT/UPDATE/DELETE para o role authenticated (idealmente restrito por e-mail/role específico do admin). Remover admin-guia-upload.html do repositório (é um trecho de referência, não deveria estar publicado como página funcional).

[A3] **ALTA** — Sem rate limiting/lockout na senha do Worker gh-proxy

Arquivo:linha

- worker-proxy/worker.js:36-43

Descrição: O Worker tatuagem-gh-proxy confere a senha do admin (ADMIN_PASSWORD) em toda chamada, mas não há nenhum limite de tentativas, atraso progressivo ou bloqueio temporário. Essa senha protege escrita completa no repositório (via GITHUB_TOKEN real, guardado só no Worker) — incluindo .github/workflows/*, que usa secrets.SUPABASE_SERVICE_ROLE_KEY (chave que ignora RLS). Ou seja, essa senha é, na prática, a chave-mestra de todo o site e, em cadeia, de parte do banco.

Impacto: Um atacante pode tentar força bruta contra ADMIN_PASSWORD sem limite algum, na velocidade que a rede/Cloudflare permitir.

Condição de explorabilidade: Nenhuma condição especial — o Worker está publicamente acessível em qualquer chamada HTTP.

Sugestão de correção: Adicionar rate limiting (Cloudflare Rate Limiting Rules, ou contagem por IP em Workers KV/Durable Objects) e usar uma senha de alta entropia — a atual já está exposta (ver A1) e tem baixa entropia.

[B1] BAIXA — Comparação de senha não constant-time no worker.js**Arquivo:linha**

- worker-proxy/worker.js:38

Descrição: A checagem ``senhaEnviada !== env.ADMIN_PASSWORD`` usa comparação padrão de string do JavaScript, que não é constant-time — em teoria, vulnerável a um timing attack para descobrir a senha caractere por caractere. Impacto prático baixo (o jitter da rede normalmente já dificulta esse tipo de ataque em um Worker exposto publicamente pela internet).

Impacto: Vetor teórico de descoberta de senha por análise de tempo de resposta.

Condição de explorabilidade: Exige medição de latência de rede muito precisa — de baixa viabilidade prática nesse contexto.

Sugestão de correção: Usar uma comparação constant-time (ex.: `crypto.subtle.timingSafeEqual`, se disponível no runtime do Worker, ou comparar hashes).

3. IDOR

[M1] MÉDIA — Enumeração de clientes via código de indicação de baixa entropia**Arquivo:linha**

- reserva.html:1132-1135 (gerarCodigo), 1194-1196 (verificar_codigo_indicacao)
- indicacoes.html:391-393 (consultar_progresso_indicacao)

Descrição: `gerarCodigo()` gera códigos no formato MT-XXXX com XXXX de 1000 a 9999 — só 9.000 combinações possíveis. Esse código funciona como token de acesso implícito para duas RPCs públicas do Supabase, sem nenhuma prova de posse: `verificar_codigo_indicacao` devolve o nome do dono do código; `consultar_progresso_indicacao` devolve nome, indicações confirmadas e status da sessão grátis. Nenhum rate limiting visível no lado do cliente.

Impacto: Um atacante pode iterar os 9.000 códigos possíveis e coletar nome + progresso de indicação de toda a base de clientes cadastrados (informação de negócio + PII leve: nome vinculado a comportamento de indicação).

Condição de explorabilidade: Nenhuma condição especial — as RPCs são chamáveis publicamente com a anon key, que já é pública.

Sugestão de correção: Aumentar a entropia do código (ex.: base36 de 6+ caracteres aleatórios de fonte criptograficamente segura) e/ou adicionar rate limiting na RPC do lado do Supabase (ex.: Edge Function com limite por IP).

[M2] MÉDIA — Oráculo de existência/dados via busca de cliente por WhatsApp**Arquivo:linha**

- reserva.html:1327-1335 (buscar_cliente_por_whatsapp, chamada após erro 23505)

Descrição: Ao tentar cadastrar um WhatsApp já existente (erro de unicidade 23505), o código automaticamente busca e expõe o registro do dono real desse número (incluindo `codigo_indicacao`) para quem quer que tenha digitado esse número no formulário — sem nenhuma prova de que o número pertence a quem está preenchendo (ex.: sem confirmação por OTP).

Impacto: Permite descobrir se um número de WhatsApp específico já é cliente cadastrado (oráculo de existência) e obter o código de indicação do dono real desse número.

Condição de explorabilidade: Requer apenas conhecer o número de WhatsApp alvo (ou testar números em sequência).

Sugestão de correção: Não devolver dados do registro existente em caso de conflito; devolver só uma mensagem genérica ('esse WhatsApp já está cadastrado, entre em contato pelo WhatsApp pra recuperar seu código') sem chamar `buscar_cliente_por_whatsapp` a partir de um número não verificado.

4. Chaves expostas

[A1] **ALTA** — Senha do painel do Guia commitada em texto puro no histórico do git

Arquivo:linha

- Histórico git (git log -p) — múltiplos commits, incluindo o valor atual em adminguia.html:224

Descrição: O valor real da ADMIN_SENHA ('Sofilhadaputa1') foi commitado em texto puro várias vezes ao longo do histórico do repositório público (confirmado via git log -p, alternando com um placeholder 'TROQUE-ESSA-SENHA'). Um comentário no próprio código ('senha simples só pra travar o acesso a essa tela — não é autenticação do Supabase') mostra que o autor já sabia que não era uma proteção real.

Impacto: Credencial permanentemente exposta em repositório público.

Condição de explorabilidade: Qualquer clone do repositório público já tem acesso ao histórico completo.

Sugestão de correção: Tratar a senha como comprometida: trocar (ou eliminar, ver C1) em todos os lugares onde for reaproveitada. Reescrever o histórico do git (git filter-repo / BFG) só reduz exposição futura, não desfaz clones já feitos — não depender disso como mitigação principal.

[I1] **INFORMATIVA** — Chave da Google Maps JavaScript API exposta no client-side

Arquivo:linha

- index.html:1945

Descrição: A chave da Google Maps JavaScript API (AlzaSyDt...) está embutida na URL do script. Esse é o padrão esperado para essa API específica do Google — ela roda no navegador por design e é protegida por restrições configuradas no Google Cloud Console (referrer HTTP + API), não por estar 'escondida'. Não é, por si, um vazamento.

Impacto: Se a chave não estiver restrita, pode ser copiada e usada por terceiros, gerando cobrança indevida na conta do Google Cloud do projeto.

Condição de explorabilidade: Depende de a chave não estar restrita no painel do Google Cloud (não verificável a partir do repositório).

Sugestão de correção: Confirmar no Google Cloud Console que a chave está restrita por referer HTTP (micaeltatuagem.com.br/*) e por API (Maps JavaScript API + Places).

5. XSS

[A4] **ALTA** — XSS armazenado via URL javascript: em link_estilo **[CORRIGIDO]**

✓ Corrigido no commit d2308a8f (link_estilo só é usado como href se começar com '/' ou 'https://').

Arquivo:linha

- guia-verbete.html:~217 (elLink.href = verbete.link_estilo)

Descrição: guia-verbete.html atribui o campo link_estilo (vindo direto do banco) a elLink.href sem validar o esquema da URL. Combinado com C1 (esse mesmo campo é gravável por qualquer um, sem autenticação), um atacante pode gravar um verbete com link_estilo = "javascript:..." — qualquer visitante que clicar no link 'ver mais' executa JavaScript arbitrário no contexto de origem do site.

Impacto: Execução de script arbitrário no navegador de visitantes do site público, a partir de um campo de banco de dados gravável sem autenticação.

Condição de explorabilidade: Depende de conseguir gravar um valor malicioso em guia_verbetes.link_estilo — ver C1.

Sugestão de correção: Validar que o valor começa com '/' (link interno) ou 'https://' antes de usar como href, rejeitando qualquer outro esquema (javascript:, data:, vbscript:). Resolver C1 elimina a via de escrita, mas a validação de esquema é defesa em profundidade independente disso.

Tabela de achados

Sev.	Arquivo:linha	Descrição
Crítica	[C1] adminguia.html:219-220, 224, 251-259, 383-386, 398-408, 448-465	Painel do Guia sem autenticação real — escrita livre em guia_verbetes e Storage [CORRIGIDO]
Alta	[A1] Histórico git	Senha do painel do Guia commitada em texto puro no histórico do git
Alta	[A2] scripts/generate-guia-pages.js: 529-533	Path traversal via slug não sanitizado em generate-guia-pages.js [CORRIGIDO]
Alta	[A3] worker-proxy/worker.js:36-43	Sem rate limiting/lockout na senha do Worker gh-proxy
Alta	[A4] guia-verbete.html:~217	XSS armazenado via URL javascript: em link_estilo [CORRIGIDO]
Média	[M1] reserva.html:1132-1135	Enumeração de clientes via código de indicação de baixa entropia
Média	[M2] reserva.html:1327-1335	Oráculo de existência/dados via busca de cliente por WhatsApp
Média	[M3] reserva.html:1321	Falta de validação de regra de negócio no INSERT direto em clientes_promo [CORRIGIDO]
Média	[M4] anamnese.html:836-861	RLS de SELECT em anamneses (CPF/RG/dados de saúde) não verificável a partir do repositório [VERIFICADO OK]
Baixa	[B1] worker-proxy/worker.js:38	Comparação de senha não constant-time no worker.js
Informativa	[I1] index.html:1945	Chave da Google Maps JavaScript API exposta no client-side

Recomendações priorizadas

Prioridade	Recomendação
P1	Corrigir C1: trocar a checagem client-side de adminguia.html por Supabase Auth real e configurar RLS de escrita em guia_verbetes restrita a authenticated; remover admin-guia-upload.html do repositório.
P1	Confirmar (A2) e sanitizar o campo slug em generate-guia-pages.js com allow-list antes de usar em path.join, independentemente da correção de C1.
P1	Verificar manualmente no dashboard do Supabase (M4) que a política de SELECT em anamneses está restrita a authenticated — dado sensível pela LGPD (CPF/RG/saúde).
P2	Adicionar rate limiting/lockout no Worker gh-proxy (A3) e trocar ADMIN_PASSWORD por um valor de alta entropia, já que o atual está exposto (A1).
P2	Validar esquema de URL antes de usar link_estilo como href em guia-verbete.html (A4), como defesa em profundidade independente de C1.
P2	Aumentar a entropia dos códigos de indicação (M1) e remover a devolução de dados de terceiros no fluxo de WhatsApp duplicado (M2).
P3	Confirmar defaults/constraints em clientes_promo (M3) para impedir que um INSERT direto sobrescreva colunas de premiação; considerar mover a lógica para uma RPC SECURITY DEFINER.
P3	Trocar a comparação de senha em worker.js por uma função constant-time (B1).
P3	Confirmar restrições de referrer/API da chave do Google Maps no Google Cloud Console (I1).

Issues para o GitHub

Texto completo de cada issue em Markdown, pronto para copiar e colar no GitHub. Achados triviais relacionados foram agrupados numa issue única para não gerar spam.

--- ISSUE 1 ---

Título: [Segurança] Painel do Guia sem autenticação real e endpoint público sem login algum

Labels: security, critical

Problema

`adminguia.html` decide se libera o painel comparando a senha digitada com uma constante no JavaScript do navegador (`ADMIN\_SENHA`, linha 224) – não há nenhuma verificação no servidor. Todas as escritas (INSERT/UPDATE/DELETE em `guia\_verbetes`, upload no Storage) usam só a chave publicável (anon) do Supabase.

`admin-guia-upload.html` é um arquivo publicado no ar que faz o mesmo INSERT/upload **sem nenhuma tela de login**, cosmética ou não.

Por que é explorável

A anon key do Supabase é pública por design (está embutida em `guia.html`, `guia-verbete.html` etc.). Sem verificação server-side, qualquer pessoa pode chamar a API REST do Supabase diretamente com essa chave e escrever na tabela – a senha em `adminguia.html` não impede nada de verdade, e `admin-guia-upload.html` nem tenta.

Evidência

- `adminguia.html:224` – `const ADMIN\_SENHA = 'Sofilhadaputa1';`
- `adminguia.html:251-259` – comparação client-side, sem chamada a `supabase.auth`
- `adminguia.html:383-386, 448-465` – DELETE/PATCH/POST em `guia\_verbetes` só com anon key
- `admin-guia-upload.html:126-135` – POST em `guia\_verbetes` sem nenhuma tela de login

Impacto

Qualquer pessoa na internet pode criar, editar ou apagar verbetes do Guia e subir arquivos no Storage sem autenticação, e ver essas mudanças publicadas automaticamente no site via a Action `generate-guia-pages.yml`.

Sugestão de correção

1. Trocar a checagem client-side por `supabase.auth.signInWithPassword()` (mesmo padrão de `admin\_promo.html`).
2. Configurar RLS em `guia\_verbetes` (e no bucket `guia-imagens`) para só permitir escrita ao role `authenticated`.
3. Remover `admin-guia-upload.html` do repositório.

Critérios de aceite

- [] `adminguia.html` usa `supabase.auth.signInWithPassword()` e nenhuma senha fica hardcoded no HTML
- [] RLS de `guia\_verbetes` bloqueia INSERT/UPDATE/DELETE para o role `anon` (testado com a anon key pública)
- [] RLS/policy do bucket `guia-imagens` bloqueia upload para `anon`
- [] `admin-guia-upload.html` removido do repositório

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

Título: [Segurança] Path traversal via slug não sanitizado em generate-guia-pages.js

Labels: security, high

Problema

`v.slug` (campo da tabela `guia\_verbetes`) é usado sem sanitização em `path.join(OUTPUT\_DIR, v.slug)` para criar diretório e escrever `index.html`.

Por que é explorável

Um `slug` como `../../algum-caminho` resolve para fora de `guia/`. Hoje isso depende de conseguir gravar um slug malicioso em `guia\_verbetes` (ver issue de autenticação do painel do Guia), mas deveria ser corrigido de forma independente, como defesa em profundidade.

```
## Evidência
`scripts/generate-guia-pages.js:529-533`
```js
const dir = path.join(OUTPUT_DIR, v.slug);
fs.mkdirSync(dir, { recursive: true });
fs.writeFileSync(path.join(dir, 'index.html'), renderPagina(v, posts));
```
```

Impacto
Escrita de arquivo em caminho arbitrário dentro do runner da GitHub Action durante a geração das páginas do Guia.

Sugestão de correção
Sanitizar `v.slug` com uma allow-list (`/^[a-z0-9-]+\$`) antes de usar no `path.join`, e idealmente adicionar um `CHECK` constraint equivalente na coluna `slug` no Supabase.

Critérios de aceite

- [] Slugs fora do padrão `[a-z0-9-]+` são rejeitados/normalizados antes da escrita em disco
- [] Teste com slug contendo `../` confirma que a página é gerada dentro de `guia/`

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

Título: [Segurança] XSS armazenado via URL javascript: em link_estilo (guia-verbete.html)

Labels: security, high

Problema
`guia-verbete.html` atribui o campo `link\_estilo` (vindo do banco) diretamente a `href` sem validar o esquema da URL.

Por que é explorável
Um valor como `link\_estilo: "javascript:..."` executa JavaScript arbitrário quando o visitante clica no link, no contexto de origem do site.

```
## Evidência
`guia-verbete.html` (função `carregarVerbetes`):
```js
elLink.href = verbete.link_estilo;
```
```

Impacto
Execução de script arbitrário no navegador de visitantes do site público.

Sugestão de correção
Validar que `link\_estilo` começa com `/` ou `https://` antes de usar como `href`, rejeitando qualquer outro esquema.

Critérios de aceite

- [] Valores com esquema `javascript:`, `data:` ou `vbscript:` são ignorados/sanitizados antes de virar `href`
- [] Teste manual com `link\_estilo = "javascript:alert(1)"` confirma que o link não executa script

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

Título: [Segurança] Sem rate limiting no Worker gh-proxy e senha de baixa entropia já exposta

Labels: security, high

Problema
O Worker `tatuagem-gh-proxy` confere `ADMIN\_PASSWORD` em toda chamada, sem nenhum rate limiting/lockout. Essa senha é, na prática, a chave-mestra de escrita de todo o repositório (via `GITHUB\_TOKEN` real guardado só no Worker).

Por que é explorável
Sem limite de tentativas, um atacante pode tentar força bruta contra a senha na velocidade que a rede

permitir.

Evidência

`worker-proxy/worker.js:36-43` – checagem de senha sem nenhum controle de taxa.

Impacto

Comprometimento da senha do Worker dá escrita completa no repositório, incluindo `.github/workflows/\*`, que usa `secrets.SUPABASE\_SERVICE\_ROLE\_KEY` (chave que ignora RLS).

Sugestão de correção

Adicionar Cloudflare Rate Limiting Rules (ou contagem por IP em KV/Durable Objects) no Worker, e trocar `ADMIN\_PASSWORD` por um valor de alta entropia (o atual já está exposto no histórico git do repositório).

Critérios de aceite

- [] Requisições repetidas com senha errada do mesmo IP passam a ser bloqueadas/atrasadas
- [] `ADMIN\_PASSWORD` trocado por valor de alta entropia, nunca commitado em texto puro

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

Título: [Segurança] IDOR: enumeração de clientes via código de indicação e oráculo de WhatsApp

Labels: security, medium

Problema

Dois pontos permitem obter dados de clientes sem prova de posse:

1. `gerarCodigo()` gera códigos `MT-XXXX` com apenas 9.000 combinações possíveis, usados como token de acesso implícito nas RPCs `verificar\_codigo\_indicacao` e `consultar\_progresso\_indicacao` (retornam nome + progresso de indicação).
2. `buscar\_cliente\_por\_whatsapp`, chamada após erro de WhatsApp duplicado em `reserva.html`, devolve o registro completo (incluindo `codigo\_indicacao`) do dono real de um número digitado por qualquer visitante.

Por que é explorável

Nenhuma das duas RPCs exige prova de que quem está consultando é o dono do código/número – só é preciso saber (ou adivinhar, dado o espaço pequeno de 9.000 códigos) o valor.

Evidência

- `reserva.html:1132-1135` (gerarCodigo), `:1194-1196` (verificar_codigo_indicacao), `:1327-1335` (buscar_cliente_por_whatsapp)
- `indicacoes.html:391-393` (consultar_progresso_indicacao)

Impacto

Coleta de nome + progresso de indicação de toda a base de clientes; confirmação de quais números de WhatsApp são clientes cadastrados.

Sugestão de correção

Aumentar a entropia do código de indicação (6+ caracteres aleatórios); parar de devolver dados de terceiros no fluxo de WhatsApp duplicado (devolver só mensagem genérica).

Critérios de aceite

- [] Códigos de indicação passam a ter espaço amostral grande o suficiente para inviabilizar enumeração
- [] Fluxo de WhatsApp duplicado não devolve mais dados de outro cliente

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

Título: [Segurança] Verificar RLS de leitura em clientes_promo e anamneses (dado sensível LGPD)

Labels: security, medium

Problema

`clientes\_promo` recebe INSERT direto do público sem validação de colunas de premiação no código-fonte do site (protegido só por RLS/constraints do lado do Supabase, não visíveis neste repositório).
`anamneses` guarda CPF, RG, endereço e respostas de saúde – dado sensível pela LGPD – e o código do

site está correto (INSERT público sem SELECT cruzado; leitura só autenticada em `adminanamnese.html`), mas a política de RLS de SELECT não pôde ser verificada a partir do repositório.

Por que é importante

Não há evidência de vulnerabilidade confirmada – mas o custo de uma política mal configurada nessas duas tabelas é alto o suficiente (fraude de premiação / vazamento de CPF+RG+saúde) para justificar verificação manual prioritária.

Evidência

- `reserva.html:1321` – INSERT direto em `clientes\_promo`
- `anamnese.html:836-861` – payload com CPF/RG/dados de saúde, só INSERT
- `adminanamnese.html:518, 529-544` – leitura só após `signInWithPassword` (correto)

Sugestão de correção

No dashboard do Supabase: confirmar que colunas de premiação em `clientes\_promo` têm DEFAULT/GENERATED que o role `anon` não pode sobrescrever; confirmar que SELECT em `anamneses` é restrito a `authenticated` (idealmente a um role/e-mail específico de admin).

Critérios de aceite

- [] Confirmado (ou corrigido) que INSERT anônimo em `clientes\_promo` não pode setar colunas de premiação
- [] Confirmado (ou corrigido) que SELECT em `anamneses` é restrito a admin autenticado

--- FIM ISSUE 6 ---